// app/api/auth/login/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import bcrypt from 'bcryptjs'
import pool from '@/lib/db'
import { signToken } from '@/lib/auth'
import { success, failure } from '@/lib/result'

export async function POST(req: NextRequest) {
  const body = await req.json()
  const { login, password } = body as { login?: string; password?: string }

  if (!login || !password) {
    return NextResponse.json(failure('username/email and password are required', 400), { status: 400 })
  }

  const result = await pool.query(
    'SELECT id, username, email, password_hash FROM users WHERE email = $1 OR username = $1',
    [login]
  )

  if (result.rows.length === 0) {
    return NextResponse.json(failure('Invalid credentials', 401), { status: 401 })
  }

  const user = result.rows[0]
  const valid = await bcrypt.compare(password, user.password_hash)

  if (!valid) {
    return NextResponse.json(failure('Invalid credentials', 401), { status: 401 })
  }

  const token = signToken({ userId: user.id, username: user.username })
  const { password_hash: _, ...userWithoutHash } = user

  const response = NextResponse.json(success({ user: userWithoutHash, token }))
  response.cookies.set('token', token, {
    httpOnly: true,
    secure: process.env.NODE_ENV === 'production',
    sameSite: 'lax',
    maxAge: 60 * 60 * 24 * 7,
  })
  return response
}

```

// app/api/auth/logout/route.ts

```
import { NextResponse } from 'next/server'
import { success } from '@/lib/result'

export async function POST() {
  const response = NextResponse.json(success({ loggedOut: true }))
  response.cookies.delete('token')
  return response
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

```
}

```

// app/api/auth/register/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import bcrypt from 'bcryptjs'
import pool from '@/lib/db'
import { signToken } from '@/lib/auth'
import { success, failure } from '@/lib/result'

export async function POST(req: NextRequest) {
  const body = await req.json()
  const { username, email, password } = body as {
    username?: string
    email?: string
    password?: string
  }

  if (!username || !email || !password) {
    return NextResponse.json(failure('username, email and password are required', 400), { status: 400 })
  }

  const passwordHash = await bcrypt.hash(password, 10)

  try {
    const result = await pool.query(
      'INSERT INTO users (username, email, password_hash) VALUES ($1, $2, $3) RETURNING id, username, email, created_at',
      [username, email, passwordHash]
    )
    const user = result.rows[0]
    const token = signToken({ userId: user.id, username: user.username })

    const response = NextResponse.json(success({ user, token }), { status: 201 })
    response.cookies.set('token', token, {
      httpOnly: true,
      secure: process.env.NODE_ENV === 'production',
      sameSite: 'lax',
      maxAge: 60 * 60 * 24 * 7,
    })
    return response
  } catch (err: unknown) {
    const pgErr = err as { code?: string }
    if (pgErr.code === '23505') {
      return NextResponse.json(failure('Username or email already exists', 409), { status: 409 })
    }
    console.error(err)
    return NextResponse.json(failure('Server error', 500), { status: 500 })
  }
}

```

// app/api/dependencies/[projectId]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import pool from '@/lib/db'
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

```
import { getAuthUser } from '@/lib/middleware'
import { success, failure } from '@/lib/result'

type Params = { params: Promise<{ projectId: string }> }

async function verifyOwnership(projectId: string, userId: string) {
  const r = await pool.query(
    'SELECT id FROM projects WHERE id = $1 AND user_id = $2',
    [projectId, userId]
  )
  return r.rows.length > 0
}

export async function POST(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: 401 })
  const { projectId } = await params

  if (!(await verifyOwnership(projectId, auth.value.userId)))
    return NextResponse.json(failure('Not found', 404), { status: 404 })

  const { from_task_id, to_task_id, type = 2, lag = 0 } = await req.json()
  if (!from_task_id || !to_task_id)
    return NextResponse.json(failure('from_task_id and to_task_id required', 400), { status: 400 })

  const r = await pool.query(
    `INSERT INTO dependencies (project_id, from_task_id, to_task_id, type, lag)
     VALUES ($1,$2,$3,$4,$5) RETURNING *`,
    [projectId, from_task_id, to_task_id, type, lag]
  )
  return NextResponse.json(success(r.rows[0]), { status: 201 })
}

export async function PUT(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: 401 })
  const { projectId } = await params

  if (!(await verifyOwnership(projectId, auth.value.userId)))
    return NextResponse.json(failure('Not found', 404), { status: 404 })

  const { id, type, lag } = await req.json()
  if (!id) return NextResponse.json(failure('id required', 400), { status: 400 })

  const r = await pool.query(
    `UPDATE dependencies
     SET type = COALESCE($1, type), lag = COALESCE($2, lag)
     WHERE id = $3 AND project_id = $4 RETURNING *`,
    [type ?? null, lag ?? null, id, projectId]
  )
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

```
  if (!r.rows[0]) return NextResponse.json(failure('Not found', 404), { status: 404 })
  return NextResponse.json(success(r.rows[0]))
}

export async function DELETE(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: 401 })
  const { projectId } = await params

  if (!(await verifyOwnership(projectId, auth.value.userId)))
    return NextResponse.json(failure('Not found', 404), { status: 404 })

  const { id } = await req.json()
  if (!id) return NextResponse.json(failure('id required', 400), { status: 400 })

  await pool.query(
    'DELETE FROM dependencies WHERE id = $1 AND project_id = $2',
    [id, projectId]
  )
  return NextResponse.json(success({ deleted: id }))
}

```

// app/api/projects/[id]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import pool from '@/lib/db'
import { getAuthUser } from '@/lib/middleware'
import { success, failure } from '@/lib/result'

type Params = { params: Promise<{ id: string }> }

export async function GET(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { id } = await params

  const result = await pool.query(
    'SELECT * FROM projects WHERE id = $1 AND user_id = $2',
    [id, auth.value.userId]
  )
  if (result.rows.length === 0) {
    return NextResponse.json(failure('Project not found', 404), { status: 404 })
  }
  return NextResponse.json(success(result.rows[0]))
}

export async function PUT(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { id } = await params

  const body = await req.json()
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

```
  const { name, start_date, end_date, status_date } = body as {
    name?: string
    start_date?: string | null
    end_date?: string | null
    status_date?: string | null
  }

  const result = await pool.query(
    `UPDATE projects
     SET name = COALESCE($1, name),
         start_date = COALESCE($2, start_date),
         end_date = COALESCE($3, end_date),
         status_date = $4
     WHERE id = $5 AND user_id = $6
     RETURNING *`,
    [name ?? null, start_date ?? null, end_date ?? null, status_date ?? null, id, auth.value.userId]
  )
  if (result.rows.length === 0) {
    return NextResponse.json(failure('Project not found', 404), { status: 404 })
  }
  return NextResponse.json(success(result.rows[0]))
}

export async function DELETE(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { id } = await params

  const result = await pool.query(
    'DELETE FROM projects WHERE id = $1 AND user_id = $2 RETURNING id',
    [id, auth.value.userId]
  )
  if (result.rows.length === 0) {
    return NextResponse.json(failure('Project not found', 404), { status: 404 })
  }
  return NextResponse.json(success({ deleted: true }))
}

```

// app/api/projects/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import pool from '@/lib/db'
import { getAuthUser } from '@/lib/middleware'
import { success, failure } from '@/lib/result'

export async function GET(req: NextRequest) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const result = await pool.query(
    'SELECT * FROM projects WHERE user_id = $1 ORDER BY created_at DESC',
    [auth.value.userId]
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

```
  )
  return NextResponse.json(success(result.rows))
}

export async function POST(req: NextRequest) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const body = await req.json()
  const { name, start_date, end_date, status_date } = body as {
    name?: string
    start_date?: string
    end_date?: string
    status_date?: string
  }

  if (!name) {
    return NextResponse.json(failure('name is required', 400), { status: 400 })
  }

  const result = await pool.query(
    'INSERT INTO projects (user_id, name, start_date, end_date, status_date) VALUES ($1, $2, $3, $4, $5) RETURNING *',
    [auth.value.userId, name, start_date ?? null, end_date ?? null, status_date ?? null]
  )
  return NextResponse.json(success(result.rows[0]), { status: 201 })
}

```

// app/api/tasks/[projectId]/[taskId]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import pool from '@/lib/db'
import { getAuthUser } from '@/lib/middleware'
import { success, failure } from '@/lib/result'

type Params = { params: Promise<{ projectId: string; taskId: string }> }

// GET /api/tasks/[projectId]/[taskId] — task detail + lifecycle events
export async function GET(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const { projectId, taskId } = await params

  const proj = await pool.query(
    'SELECT id FROM projects WHERE id = $1 AND user_id = $2',
    [projectId, auth.value.userId]
  )
  if (!proj.rows.length)
    return NextResponse.json(failure('Not found', 404), { status: 404 })

  const [taskRes, lifecycleRes] = await Promise.all([
    pool.query('SELECT * FROM tasks WHERE id = $1 AND project_id = $2', [taskId, projectId]),
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

```
    pool.query(
      `SELECT l.*, u.username AS created_by_name
       FROM task_lifecycle l
       LEFT JOIN users u ON u.id = l.created_by
       WHERE l.task_id = $1
       ORDER BY l.created_at DESC`,
      [taskId]
    ),
  ])

  if (!taskRes.rows.length)
    return NextResponse.json(failure('Task not found', 404), { status: 404 })

  return NextResponse.json(success({
    task: taskRes.rows[0],
    lifecycle: lifecycleRes.rows,
  }))
}

```

// app/api/tasks/[projectId]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import type { PoolClient } from 'pg'
import pool from '@/lib/db'
import { getAuthUser } from '@/lib/middleware'
import { success, failure } from '@/lib/result'

type Params = { params: Promise<{ projectId: string }> }

async function verifyProjectOwnership(projectId: string, userId: string): Promise<boolean> {
  const result = await pool.query(
    'SELECT id FROM projects WHERE id = $1 AND user_id = $2',
    [projectId, userId]
  )
  return result.rows.length > 0
}

// Generate next task_code for a project: T-001, T-002, ...
async function nextTaskCode(client: PoolClient, projectId: string): Promise<string> {
  const res = await client.query(
    `SELECT COALESCE(MAX(CAST(SUBSTRING(task_code FROM 3) AS INTEGER)), 0) + 1 AS seq
     FROM tasks WHERE project_id = $1 AND task_code IS NOT NULL`,
    [projectId]
  )
  const seq = res.rows[0].seq as number
  return `T-${String(seq).padStart(3, '0')}`
}

// Insert a lifecycle event
async function addLifecycle(
  client: PoolClient,
  opts: {
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
    taskId: string; taskCode: string; projectId: string; userId: string
    event_type: string; field_name?: string | null
    old_value?: string | null; new_value?: string | null
    description: string
  }
) {
  await client.query(
    `INSERT INTO task_lifecycle
       (task_id, task_code, project_id, event_type, field_name, old_value, new_value, description, created_by)
     VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9)`,
    [opts.taskId, opts.taskCode, opts.projectId, opts.event_type,
     opts.field_name ?? null, opts.old_value ?? null, opts.new_value ?? null,
     opts.description, opts.userId]
  )
}

const FIELD_LABELS: Record<string, string> = {
  name: '任务名称', start_date: '开始日期', end_date: '结束日期',
  duration: '工期', assignee: '责任人', percent_done: '完成度',
  parent_id: '父任务', is_milestone: '里程碑', note: '备注',
}

function fmtFieldVal(field: string, val: unknown): string {
  if (val === null || val === undefined || val === '') return '(空)'
  if (field === 'start_date' || field === 'end_date') {
    const s = String(val); return s.includes('T') ? s.split('T')[0] : s
  }
  if (field === 'duration') return `${val} 天`
  if (field === 'percent_done') return `${val}%`
  if (field === 'is_milestone') return val ? '是' : '否'
  return String(val)
}

// ── GET ──────────────────────────────────────────────────────────────
export async function GET(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { projectId } = await params

  const owned = await verifyProjectOwnership(projectId, auth.value.userId)
  if (!owned) return NextResponse.json(failure('Project not found', 404), { status: 404 })

  const [tasks, deps] = await Promise.all([
    pool.query(
      'SELECT * FROM tasks WHERE project_id = $1 AND is_deleted = false ORDER BY order_index ASC',
      [projectId]
    ),
    // Exclude deps where either endpoint is soft-deleted
    pool.query(
      `SELECT d.* FROM dependencies d
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
        JOIN tasks ft ON ft.id = d.from_task_id AND ft.is_deleted = false
        JOIN tasks tt ON tt.id = d.to_task_id   AND tt.is_deleted = false
        WHERE d.project_id = $1`,
       [projectId]
      ),
    ])

  return NextResponse.json(success({ tasks: tasks.rows, dependencies: deps.rows }))
}

// ── POST ──────────────────────────────────────────────────────────────
export async function POST(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { projectId } = await params

  const owned = await verifyProjectOwnership(projectId, auth.value.userId)
  if (!owned) return NextResponse.json(failure('Project not found', 404), { status: 404 })

  const body = await req.json()
  const taskInputs = Array.isArray(body) ? body : [body]

  const client = await pool.connect()
  const inserted = []
  try {
    await client.query('BEGIN')
    for (const task of taskInputs) {
      const { name, parent_id, assignee, start_date, end_date, duration, duration_unit,
              percent_done, is_milestone, note, order_index } = task as Record<string, unknown>
      if (!name) continue

      const code = await nextTaskCode(client, projectId)

      const r = await client.query(
        `INSERT INTO tasks
           (project_id, parent_id, task_code, name, assignee, start_date, end_date,
            duration, duration_unit, percent_done, is_milestone, note, order_index)
         VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13) RETURNING *`,
        [projectId, parent_id ?? null, code, name,
         assignee ?? null, start_date ?? null, end_date ?? null,
         duration ?? null, duration_unit ?? 'day',
         percent_done ?? 0, is_milestone ?? false, note ?? null, order_index ?? 0]
      )
      const newTask = r.rows[0]
      inserted.push(newTask)

      await addLifecycle(client, {
        taskId: newTask.id, taskCode: code, projectId,
        userId: auth.value.userId, event_type: 'created',
        description: `创建任务「${name}」（${code}）`,
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
      })
    }
    await client.query('COMMIT')
  } catch (err) {
    await client.query('ROLLBACK')
    throw err
  } finally {
    client.release()
  }

  return NextResponse.json(success(inserted), { status: 201 })
}

// ── PUT ──────────────────────────────────────────────────────────────
export async function PUT(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { projectId } = await params

  const owned = await verifyProjectOwnership(projectId, auth.value.userId)
  if (!owned) return NextResponse.json(failure('Project not found', 404), { status: 404 })

  const body = await req.json()
  const taskInputs = Array.isArray(body) ? body : [body]

  const client = await pool.connect()
  const updated = []
  try {
    await client.query('BEGIN')

    for (const task of taskInputs) {
      const { id } = task as { id: string }
      if (!id) continue

      // Fetch current state for diff
      const prev = await client.query(
        'SELECT * FROM tasks WHERE id = $1 AND project_id = $2 AND is_deleted = false',
        [id, projectId]
      )
      if (!prev.rows.length) continue
      const old = prev.rows[0]

      const hasParentKey = 'parent_id' in task
      const hasAssigneeKey = 'assignee' in task

      const r = await client.query(
        `UPDATE tasks SET
           name         = COALESCE($1, name),
           parent_id    = CASE WHEN $13 THEN $2 ELSE parent_id END,
           assignee     = CASE WHEN $14 THEN $3 ELSE assignee END,
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
            start_date   = COALESCE($4, start_date),
            end_date     = COALESCE($5, end_date),
            duration     = COALESCE($6, duration),
            duration_unit= COALESCE($7, duration_unit),
            percent_done = COALESCE($8, percent_done),
            is_milestone = COALESCE($9, is_milestone),
            note         = $10,
            order_index  = COALESCE($11, order_index),
            updated_at   = NOW()
          WHERE id = $12 AND project_id = $15
          RETURNING *`,
         [
           (task as Record<string, unknown>).name ?? null,
           (task as Record<string, unknown>).parent_id ?? null,
           (task as Record<string, unknown>).assignee ?? null,
           (task as Record<string, unknown>).start_date ?? null,
           (task as Record<string, unknown>).end_date ?? null,
           (task as Record<string, unknown>).duration ?? null,
           (task as Record<string, unknown>).duration_unit ?? null,
           (task as Record<string, unknown>).percent_done ?? null,
           (task as Record<string, unknown>).is_milestone ?? null,
           (task as Record<string, unknown>).note ?? null,
           (task as Record<string, unknown>).order_index ?? null,
           id, hasParentKey, hasAssigneeKey, projectId,
         ]
       )
       if (!r.rows[0]) continue
       const cur = r.rows[0]
       updated.push(cur)

       const code: string = cur.task_code ?? old.task_code ?? id.slice(0, 8)

       // ── Detect position move (order_index or parent_id changed) ────────
       const parentChanged = hasParentKey && String(old.parent_id ?? '') !== String(cur.parent_id ?? '')
       const orderChanged  = (task as Record<string, unknown>).order_index !== undefined
                          && old.order_index !== cur.order_index

       if (parentChanged || orderChanged) {
         let desc: string
         if (parentChanged && cur.parent_id === null) {
           desc = `任务「${cur.name}」（${code}）升级为顶级任务`
         } else if (parentChanged) {
           // look up new parent name
           const parentRes = await client.query('SELECT name, task_code FROM tasks WHERE id = $1', [cur.parent_id])
           const pName = parentRes.rows[0]?.name ?? cur.parent_id
           const pCode = parentRes.rows[0]?.task_code ?? ''
           desc = `任务「${cur.name}」（${code}）移至子任务，父级：「${pName}」（${pCode}）`
         } else {
           const dir = cur.order_index < old.order_index ? '上移' : '下移'
           desc = `任务「${cur.name}」（${code}）${dir}（排序 ${old.order_index} → ${cur.order_index}）`
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
          }
          await addLifecycle(client, {
            taskId: id, taskCode: code, projectId, userId: auth.value.userId,
            event_type: 'moved',
            field_name: parentChanged ? 'parent_id' : 'order_index',
            old_value: parentChanged ? String(old.parent_id ?? '顶级') : String(old.order_index),
            new_value: parentChanged ? String(cur.parent_id ?? '顶级') : String(cur.order_index),
            description: desc,
          })
        }

        // ── Detect field updates ──────────────────────────────────────
        const COMPARE_FIELDS = ['name','start_date','end_date','duration','assignee','percent_done','is_milestone','note']
        for (const f of COMPARE_FIELDS) {
          if (!(f in task)) continue
          const norm = (v: unknown) => {
            if (v === null || v === undefined) return ''
            const s = String(v); return s.includes('T') ? s.split('T')[0] : s
          }
          const ov = norm(old[f])
          const nv = norm(cur[f])
          if (ov === nv) continue
          const label = FIELD_LABELS[f] ?? f
          const ovStr = fmtFieldVal(f, old[f])
          const nvStr = fmtFieldVal(f, cur[f])
          let desc: string
          if (f === 'duration') {
            const diff = Number(cur[f] ?? 0) - Number(old[f] ?? 0)
            desc = diff > 0
              ? `任务「${cur.name}」（${code}）工期延长 ${diff} 天（${ovStr} → ${nvStr}）`
              : `任务「${cur.name}」（${code}）工期缩短 ${Math.abs(diff)} 天（${ovStr} → ${nvStr}）`
          } else if (f === 'name') {
            desc = `任务（${code}）重命名：「${ovStr}」→「${nvStr}」`
          } else {
            desc = `任务「${cur.name}」（${code}）${label}: ${ovStr} → ${nvStr}`
          }
          await addLifecycle(client, {
            taskId: id, taskCode: code, projectId, userId: auth.value.userId,
            event_type: 'updated', field_name: f,
            old_value: ovStr, new_value: nvStr, description: desc,
          })
        }
      }

      await client.query('COMMIT')
    } catch (err) {
      await client.query('ROLLBACK')
      throw err
    } finally {
      client.release()
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
  }

  return NextResponse.json(success(updated))
}

// ── DELETE (soft) ──────────────────────────────────────────────────────
export async function DELETE(req: NextRequest, { params }: Params) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })
  const { projectId } = await params

  const owned = await verifyProjectOwnership(projectId, auth.value.userId)
  if (!owned) return NextResponse.json(failure('Project not found', 404), { status: 404 })

  const body = await req.json()
  const ids: string[] = Array.isArray(body.ids) ? body.ids : [body.id]
  if (!ids.length) return NextResponse.json(failure('No IDs provided', 400), { status: 400 })

  const client = await pool.connect()
  try {
    await client.query('BEGIN')

    // Collect all descendants recursively for cascade soft-delete
    const allIds = new Set<string>(ids)
    let frontier = [...ids]
    while (frontier.length) {
      const ph = frontier.map((_, i) => `$${i + 1}`).join(',')
      const children = await client.query(
        `SELECT id FROM tasks WHERE parent_id IN (${ph}) AND is_deleted = false`,
        frontier
      )
      const childIds = children.rows.map((r: { id: string }) => r.id).filter((id: string) => !allIds.has(id))
      childIds.forEach((id: string) => allIds.add(id))
      frontier = childIds
    }

    const allIdArr = [...allIds]
    const ph = allIdArr.map((_, i) => `$${i + 2}`).join(',')

    // Fetch tasks for lifecycle before soft-deleting
    const toDelete = await client.query(
      `SELECT id, task_code, name FROM tasks WHERE project_id = $1 AND id IN (${ph})`,
      [projectId, ...allIdArr]
    )

    // Soft-delete
    await client.query(
      `UPDATE tasks SET is_deleted = true, deleted_at = NOW()
       WHERE project_id = $1 AND id IN (${ph})`,
      [projectId, ...allIdArr]
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

```
    )

    // Lifecycle events
    for (const t of toDelete.rows) {
      await addLifecycle(client, {
        taskId: t.id, taskCode: t.task_code ?? t.id.slice(0, 8),
        projectId, userId: auth.value.userId,
        event_type: 'deleted',
        description: `删除任务「${t.name}」（${t.task_code ?? ''}）`,
      })
    }

    await client.query('COMMIT')
    return NextResponse.json(success({ deleted: allIdArr }))
  } catch (err) {
    await client.query('ROLLBACK')
    throw err
  } finally {
    client.release()
  }
}

```

// app/api/versions/[projectId]/[versionId]/route.ts

```
import { NextRequest, NextResponse } from 'next/server'
import pool from '@/lib/db'
import { getAuthUser } from '@/lib/middleware'

type Params = { projectId: string; versionId: string }

// GET /api/versions/[projectId]/[versionId] — get snapshot data
export async function GET(req: NextRequest, { params }: { params: Promise<Params> }) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const { projectId, versionId } = await params
  const client = await pool.connect()
  try {
    const res = await client.query(
      `SELECT v.* FROM project_versions v
       JOIN projects p ON p.id = v.project_id
       WHERE v.id = $1 AND v.project_id = $2 AND p.user_id = $3`,
      [versionId, projectId, auth.value.userId],
    )
    if (!res.rows.length)
      return NextResponse.json({ ok: false, error: 'Not found', code: 404 }, { status: 404 })

    return NextResponse.json({ ok: true, value: res.rows[0].snapshot })
  } finally {
    client.release()
  }
}
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

```

// POST /api/versions/[projectId]/[versionId]/restore — restore version
export async function POST(req: NextRequest, { params }: { params: Promise<Params> }) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const { projectId, versionId } = await params
  const client = await pool.connect()
  try {
    await client.query('BEGIN')

    const res = await client.query(
      `SELECT v.snapshot FROM project_versions v
       JOIN projects p ON p.id = v.project_id
       WHERE v.id = $1 AND v.project_id = $2 AND p.user_id = $3`,
      [versionId, projectId, auth.value.userId],
    )
    if (!res.rows.length) {
      await client.query('ROLLBACK')
      return NextResponse.json({ ok: false, error: 'Not found', code: 404 }, { status: 404 })
    }

    const { tasks, dependencies } = res.rows[0].snapshot as { tasks: Record<string, unknown>[]; dependencies: Record<string,

    // Wipe current tasks + deps (cascade removes deps too)
    await client.query('DELETE FROM tasks WHERE project_id = $1', [projectId])

    // Re-insert tasks (two passes: parents first, then children)
    const sorted = [...tasks].sort((a, b) => {
      if (!a.parent_id && b.parent_id) return -1
      if (a.parent_id && !b.parent_id) return 1
      return 0
    })
    for (const t of sorted) {
      await client.query(
        `INSERT INTO tasks
           (id, project_id, parent_id, name, assignee, start_date, end_date,
            duration, duration_unit, percent_done, is_milestone, note, order_index,
            created_at, updated_at)
         VALUES ($1,$2,$3,$4,$5,$6,$7,$8,$9,$10,$11,$12,$13,$14,$15)
         ON CONFLICT (id) DO UPDATE SET
           name=EXCLUDED.name, assignee=EXCLUDED.assignee,
           start_date=EXCLUDED.start_date, end_date=EXCLUDED.end_date,
           duration=EXCLUDED.duration, duration_unit=EXCLUDED.duration_unit,
           percent_done=EXCLUDED.percent_done, is_milestone=EXCLUDED.is_milestone,
           note=EXCLUDED.note, order_index=EXCLUDED.order_index,
           updated_at=NOW()`,
        [
          t.id, projectId, t.parent_id ?? null, t.name, t.assignee ?? null,
          t.start_date ?? null, t.end_date ?? null,
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

```
751             t.duration ?? null, t.duration_unit ?? 'day',
752             t.percent_done ?? 0, t.is_milestone ?? false,
753             t.note ?? null, t.order_index ?? 0,
754             t.created_at, t.updated_at,
755           ],
756         )
757       }
758
759       // Re-insert dependencies
760       for (const d of dependencies) {
761         await client.query(
762           `INSERT INTO dependencies (id, project_id, from_task_id, to_task_id, type, lag)
763            VALUES ($1,$2,$3,$4,$5,$6)
764            ON CONFLICT (id) DO NOTHING`,
765           [d.id, projectId, d.from_task_id, d.to_task_id, d.type ?? 2, d.lag ?? 0],
766         )
767       }
768
769       await client.query('COMMIT')
770       return NextResponse.json({ ok: true, value: { tasks, dependencies } })
771     } catch (err) {
772       await client.query('ROLLBACK')
773       console.error('Restore failed', err)
774       return NextResponse.json({ ok: false, error: 'Restore failed' }, { status: 500 })
775     } finally {
776       client.release()
777     }
778   }
779
```

// app/api/versions/[projectId]/route.ts

```
780   import { NextRequest, NextResponse } from 'next/server'
781   import pool from '@/lib/db'
782   import { getAuthUser } from '@/lib/middleware'
783
784   // ── Human-readable field names ──────────────────────────────────────
785   const FIELD_LABELS: Record<string, string> = {
786     name:         '任务名称',
787     start_date:   '开始日期',
788     end_date:     '结束日期',
789     duration:     '工期',
790     assignee:     '责任人',
791     percent_done: '完成度',
792     parent_id:    '层级',
793     is_milestone: '里程碑',
794     note:         '备注',
795   }
796
797   function fmtVal(field: string, val: unknown): string {
798     if (val === null || val === undefined || val === '') return '(空)'
799     if (field === 'start_date' || field === 'end_date') {
800       const s = String(val)
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
    return s.includes('T') ? s.split('T')[0] : s
  }
  if (field === 'duration') return `${val} 天`
  if (field === 'percent_done') return `${val}%`
  if (field === 'is_milestone') return val ? '是' : '否'
  return String(val)
}

type TaskRow = Record<string, unknown>
type DepRow  = Record<string, unknown>

interface ChangeEntry {
  change_type: string
  task_name:   string | null
  field_name:  string | null
  old_value:   string | null
  new_value:   string | null
  description: string
}

// Compute depth-first flat order (same as client-side), returns ordered task IDs
function getFlatOrder(tasks: TaskRow[]): string[] {
  const kids: Record<string, TaskRow[]> = {}
  for (const t of tasks) {
    const k = (t.parent_id ?? '__root__') as string
    if (!kids[k]) kids[k] = []
    kids[k].push(t)
  }
  const order: string[] = []
  function walk(pid: string | null) {
    const key = pid ?? '__root__'
    ;(kids[key] ?? [])
      .sort((a, b) => (a.order_index as number) - (b.order_index as number))
      .forEach(t => { order.push(t.id as string); walk(t.id as string) })
  }
  walk(null)
  return order
}

function diffSnapshots(
  prevTasks: TaskRow[], prevDeps: DepRow[],
  currTasks: TaskRow[], currDeps: DepRow[],
): ChangeEntry[] {
  const changes: ChangeEntry[] = []

  const prevMap = new Map(prevTasks.map(t => [t.id as string, t]))
  const currMap = new Map(currTasks.map(t => [t.id as string, t]))

  // Added tasks
  for (const t of currTasks) {
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
      if (!prevMap.has(t.id as string)) {
        changes.push({
          change_type: 'task_add',
          task_name:   t.name as string,
          field_name:  null, old_value: null, new_value: null,
          description: `新增任务「${t.name}」`,
        })
      }
    }

    // Deleted tasks
    for (const t of prevTasks) {
      if (!currMap.has(t.id as string)) {
        changes.push({
          change_type: 'task_delete',
          task_name:   t.name as string,
          field_name:  null, old_value: null, new_value: null,
          description: `删除任务「${t.name}」`,
        })
      }
    }

    // Updated tasks
    const COMPARE_FIELDS = ['name','start_date','end_date','duration','assignee','percent_done','parent_id','is_milestone','no
    for (const curr of currTasks) {
      const prev = prevMap.get(curr.id as string)
      if (!prev) continue
      for (const f of COMPARE_FIELDS) {
        const pv = prev[f] ?? null
        const cv = curr[f] ?? null
        // Normalise dates to YYYY-MM-DD for comparison
        const norm = (v: unknown) => {
          if (!v) return null
          const s = String(v)
          return s.includes('T') ? s.split('T')[0] : s
        }
        const pvN = (f === 'start_date' || f === 'end_date') ? norm(pv) : pv
        const cvN = (f === 'start_date' || f === 'end_date') ? norm(cv) : cv
        if (String(pvN ?? '') !== String(cvN ?? '')) {
          const label = FIELD_LABELS[f] ?? f
          const pStr  = fmtVal(f, pv)
          const cStr  = fmtVal(f, cv)
          let desc: string
          if (f === 'duration') {
            const diff = Number(cv ?? 0) - Number(pv ?? 0)
            desc = diff > 0
              ? `任务「${curr.name}」工期延长 ${diff} 天（${pStr} → ${cStr}）`
              : `任务「${curr.name}」工期缩短 ${Math.abs(diff)} 天（${pStr} → ${cStr}）`
          } else if (f === 'start_date') {
            desc = `任务「${curr.name}」开始日期从 ${pStr} 改为 ${cStr}`
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
          } else if (f === 'end_date') {
            desc = `任务「${curr.name}」结束日期从 ${pStr} 改为 ${cStr}`
          } else if (f === 'name') {
            desc = `任务重命名: 「${pStr}」→「${cStr}」`
          } else if (f === 'parent_id') {
            desc = cv
              ? `任务「${curr.name}」设为子任务`
              : `任务「${curr.name}」升级为顶级任务`
          } else {
            desc = `任务「${curr.name}」${label}: ${pStr} → ${cStr}`
          }
          changes.push({
            change_type: 'task_update',
            task_name:   curr.name as string,
            field_name:  f,
            old_value:   pStr,
            new_value:   cStr,
            description: desc,
          })
        }
      }
    }

    // ── Position changes (reorder / swap) ──────────────────────────────
    // Only consider tasks present in both versions; ignore added/deleted tasks here
    const bothIds = new Set([...prevMap.keys()].filter(id => currMap.has(id)))

    // Compute flat tree row number restricted to shared tasks only
    // (filtering out added/deleted keeps numbering stable across the two snapshots)
    const prevShared = getFlatOrder(prevTasks).filter(id => bothIds.has(id))
    const currShared = getFlatOrder(currTasks).filter(id => bothIds.has(id))

    const prevPos: Record<string, number> = {}
    const currPos: Record<string, number> = {}
    prevShared.forEach((id, i) => { prevPos[id] = i + 1 })
    currShared.forEach((id, i) => { currPos[id] = i + 1 })

    const moved = [...bothIds].filter(id => prevPos[id] !== currPos[id])

    // Detect pure swaps: A was at p→q, B was at q→p
    const swapped = new Set<string>()
    for (let i = 0; i < moved.length; i++) {
      const idA = moved[i]
      if (swapped.has(idA)) continue
      for (let j = i + 1; j < moved.length; j++) {
        const idB = moved[j]
        if (swapped.has(idB)) continue
        if (prevPos[idA] === currPos[idB] && prevPos[idB] === currPos[idA]) {
          const nameA = currMap.get(idA)!.name as string
          const nameB = currMap.get(idB)!.name as string
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
        changes.push({
          change_type: 'task_reorder',
          task_name:   nameA,
          field_name:  null,
          old_value:   String(prevPos[idA]),
          new_value:   String(currPos[idA]),
          description: `任务「${nameA}」与「${nameB}」交换了位置（第${prevPos[idA]}行 ⇔ 第${prevPos[idB]}行）`,
        })
        swapped.add(idA)
        swapped.add(idB)
        break
      }
    }
  }

  // Remaining non-swap moves
  for (const id of moved) {
    if (swapped.has(id)) continue
    const name = currMap.get(id)!.name as string
    const from = prevPos[id]
    const to   = currPos[id]
    changes.push({
      change_type: 'task_reorder',
      task_name:   name,
      field_name:  null,
      old_value:   String(from),
      new_value:   String(to),
      description: `任务「${name}」${to < from ? '上移' : '下移'}（第${from}行 → 第${to}行）`,
    })
  }

  // Dependencies
  const prevDepSet = new Set(prevDeps.map(d => `${d.from_task_id}→${d.to_task_id}`))
  const currDepSet = new Set(currDeps.map(d => `${d.from_task_id}→${d.to_task_id}`))
  const taskNameMap = new Map(currTasks.map(t => [t.id as string, t.name as string]))
  // fallback to prev names for deleted tasks
  for (const t of prevTasks) if (!taskNameMap.has(t.id as string)) taskNameMap.set(t.id as string, t.name as string)

  for (const d of currDeps) {
    const key = `${d.from_task_id}→${d.to_task_id}`
    if (!prevDepSet.has(key)) {
      const from = taskNameMap.get(d.from_task_id as string) ?? '未知'
      const to   = taskNameMap.get(d.to_task_id   as string) ?? '未知'
      changes.push({
        change_type: 'dep_add',
        task_name: null, field_name: null, old_value: null, new_value: null,
        description: `新增依赖: 「${from}」→「${to}」`,
      })
    }
  }
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
  for (const d of prevDeps) {
    const key = `${d.from_task_id}→${d.to_task_id}`
    if (!currDepSet.has(key)) {
      const from = taskNameMap.get(d.from_task_id as string) ?? '未知'
      const to   = taskNameMap.get(d.to_task_id   as string) ?? '未知'
      changes.push({
        change_type: 'dep_delete',
        task_name: null, field_name: null, old_value: null, new_value: null,
        description: `删除依赖: 「${from}」→「${to}」`,
      })
    }
  }

  return changes
}

// GET /api/versions/[projectId] — list versions with change logs
export async function GET(req: NextRequest, { params }: { params: Promise<{ projectId: string }> }) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const { projectId } = await params
  const client = await pool.connect()
  try {
    const proj = await client.query(
      'SELECT id FROM projects WHERE id = $1 AND user_id = $2',
      [projectId, auth.value.userId],
    )
    if (!proj.rows.length)
      return NextResponse.json({ ok: false, error: 'Not found', code: 404 }, { status: 404 })

    const versRes = await client.query(
      `SELECT v.id, v.project_id, v.version_number, v.created_at,
              u.username AS created_by_name,
              jsonb_array_length(v.snapshot->'tasks') AS task_count
       FROM project_versions v
       LEFT JOIN users u ON u.id = v.created_by
       WHERE v.project_id = $1
       ORDER BY v.version_number DESC`,
      [projectId],
    )

    // Attach change logs to each version
    const versionIds = versRes.rows.map(v => v.id)
    let logsMap: Record<string, unknown[]> = {}
    if (versionIds.length > 0) {
      const logsRes = await client.query(
        `SELECT * FROM change_logs WHERE version_id = ANY($1) ORDER BY created_at ASC`,
        [versionIds],
      )
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
      logsMap = logsRes.rows.reduce((acc, l) => {
        if (!acc[l.version_id]) acc[l.version_id] = []
        acc[l.version_id].push(l)
        return acc
      }, {} as Record<string, unknown[]>)
    }

    const value = versRes.rows.map(v => ({ ...v, changes: logsMap[v.id] ?? [] }))
    return NextResponse.json({ ok: true, value })
  } finally {
    client.release()
  }
}

// POST /api/versions/[projectId] — auto-save new version with diff
export async function POST(req: NextRequest, { params }: { params: Promise<{ projectId: string }> }) {
  const auth = getAuthUser(req)
  if (!auth.ok) return NextResponse.json(auth, { status: auth.code ?? 401 })

  const { projectId } = await params

  const client = await pool.connect()
  try {
    await client.query('BEGIN')

    const proj = await client.query(
      'SELECT id FROM projects WHERE id = $1 AND user_id = $2',
      [projectId, auth.value.userId],
    )
    if (!proj.rows.length) {
      await client.query('ROLLBACK')
      return NextResponse.json({ ok: false, error: 'Not found', code: 404 }, { status: 404 })
    }

    // Current state
    const [tasksRes, depsRes] = await Promise.all([
      client.query('SELECT * FROM tasks WHERE project_id = $1 ORDER BY order_index', [projectId]),
      client.query('SELECT * FROM dependencies WHERE project_id = $1', [projectId]),
    ])
    const snapshot = { tasks: tasksRes.rows, dependencies: depsRes.rows }

    // Previous version snapshot for diffing
    const prevRes = await client.query(
      `SELECT snapshot FROM project_versions WHERE project_id = $1 ORDER BY version_number DESC LIMIT 1`,
      [projectId],
    )
    const prevSnap = prevRes.rows[0]?.snapshot as { tasks: TaskRow[]; dependencies: DepRow[] } | undefined
    const changes = prevSnap
      ? diffSnapshots(prevSnap.tasks, prevSnap.dependencies, tasksRes.rows, depsRes.rows)
      : tasksRes.rows.map(t => ({
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

```
            change_type: 'task_add' as const,
            task_name:   t.name as string,
            field_name:  null, old_value: null, new_value: null,
            description: `新增任务「${t.name}」`,
          }))

      // Next version number
      const maxRes = await client.query(
        'SELECT COALESCE(MAX(version_number), 0) AS mx FROM project_versions WHERE project_id = $1',
        [projectId],
      )
      const nextVersion = (maxRes.rows[0].mx as number) + 1

      const userRes = await client.query('SELECT username FROM users WHERE id = $1', [auth.value.userId])
      const username = userRes.rows[0]?.username ?? null

      const ins = await client.query(
        `INSERT INTO project_versions (project_id, version_number, snapshot, created_by)
         VALUES ($1, $2, $3, $4)
         RETURNING id, project_id, version_number, created_at`,
        [projectId, nextVersion, JSON.stringify(snapshot), auth.value.userId],
      )
      const versionId = ins.rows[0].id

      // Insert change logs
      for (const c of changes) {
        await client.query(
          `INSERT INTO change_logs (project_id, version_id, change_type, task_name, field_name, old_value, new_value, descript
           VALUES ($1,$2,$3,$4,$5,$6,$7,$8)`,
          [projectId, versionId, c.change_type, c.task_name, c.field_name, c.old_value, c.new_value, c.description],
        )
      }

      await client.query('COMMIT')
      return NextResponse.json({
        ok: true,
        value: { ...ins.rows[0], task_count: tasksRes.rows.length, created_by_name: username, changes },
      })
    } catch (err) {
      await client.query('ROLLBACK')
      console.error('Save version failed', err)
      return NextResponse.json({ ok: false, error: 'Save failed' }, { status: 500 })
    } finally {
      client.release()
    }
  }

```

// app/dashboard/page.tsx

```
'use client'

```

```
import { useEffect, useState } from 'react'
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

```
import { useRouter } from 'next/navigation'
import { useAppDispatch, useAppSelector } from '@/store/hooks'
import { setProjects, addProject } from '@/store/slices/projectSlice'
import ProjectCard from '@/components/ProjectCard'
import { logout } from '@/store/slices/authSlice'

export default function DashboardPage() {
  const dispatch = useAppDispatch()
  const router = useRouter()
  const { user } = useAppSelector(s => s.auth)
  const { projects } = useAppSelector(s => s.project)
  const [creating, setCreating] = useState(false)
  const [newProjectName, setNewProjectName] = useState('')
  const [loading, setLoading] = useState(true)

  useEffect(() => {
    if (!user) {
      router.push('/login')
      return
    }
    fetch('/api/projects')
      .then(r => r.json())
      .then(data => {
        if (data.ok) dispatch(setProjects(data.value))
        setLoading(false)
      })
  }, [user, dispatch, router])

  const handleCreate = async (e: React.FormEvent) => {
    e.preventDefault()
    if (!newProjectName.trim()) return

    const res = await fetch('/api/projects', {
      method: 'POST',
      headers: { 'Content-Type': 'application/json' },
      body: JSON.stringify({
        name: newProjectName,
        start_date: new Date().toISOString().split('T')[0],
      }),
    })
    const data = await res.json()
    if (data.ok) {
      dispatch(addProject(data.value))
      setNewProjectName('')
      setCreating(false)
    }
  }

  const handleLogout = async () => {
```

```
1200      await fetch('/api/auth/logout', { method: 'POST' }).catch(() => null)
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

```
      dispatch(logout())
      router.push('/login')
    }

    if (!user) return null

    return (
      <div className="min-h-screen bg-gray-50">
        <header className="bg-white border-b border-gray-200 px-6 py-4 flex items-center justify-between">
          <h1 className="text-xl font-bold text-gray-900">甘特图管理</h1>
          <div className="flex items-center gap-4">
            <span className="text-sm text-gray-600">你好，{user.username}</span>
            <button onClick={handleLogout} className="text-sm text-gray-500 hover:text-gray-700">
              退出登录
            </button>
          </div>
        </header>
        <main className="max-w-4xl mx-auto px-6 py-8">
          <div className="flex items-center justify-between mb-6">
            <h2 className="text-lg font-semibold text-gray-800">我的项目</h2>
            <button
              onClick={() => setCreating(true)}
              className="px-4 py-2 bg-blue-600 text-white text-sm rounded hover:bg-blue-700"
            >
              + 新建项目
            </button>
          </div>

          {creating && (
            <form onSubmit={handleCreate} className="mb-6 flex gap-2">
              <input
                autoFocus
                type="text"
                placeholder="项目名称"
                value={newProjectName}
                onChange={e => setNewProjectName(e.target.value)}
                className="flex-1 border border-gray-300 rounded px-3 py-2 text-sm focus:outline-none focus:ring-2 focus:ring-
              />
              <button
                type="submit"
                className="px-4 py-2 bg-blue-600 text-white text-sm rounded hover:bg-blue-700"
              >
                创建
              </button>
              <button
                type="button"
                onClick={() => setCreating(false)}
                className="px-4 py-2 bg-gray-200 text-gray-700 text-sm rounded hover:bg-gray-300"
              >
```

1250 取消

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

```
              </button>
            </form>
          )}

          {loading ? (
            <div className="text-center text-gray-500 py-12">加载中...</div>
          ) : projects.length === 0 ? (
            <div className="text-center text-gray-500 py-12">
              暂无项目，点击「新建项目」开始
            </div>
          ) : (
            <div className="grid gap-4 sm:grid-cols-2 lg:grid-cols-3">
              {projects.map(p => (
                <ProjectCard key={p.id} project={p} />
              ))}
            </div>
          )}
        </main>
      </div>
    )
  }

```

// app/globals.css

```
@import "tailwindcss";

:root {
  --background: #ffffff;
  --foreground: #171717;
}

body {
  background: var(--background);
  color: var(--foreground);
  font-family: Arial, Helvetica, sans-serif;
}

* {
  box-sizing: border-box;
}

```

// app/layout.tsx

```
import type { Metadata } from 'next'
import './globals.css'
import { StoreProvider } from '@/store/provider'

export const metadata: Metadata = {
  title: '跟踪甘特图',
```

```
  description: '项目进度甘特图管理工具',
}

export default function RootLayout({
  children,
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

```
}: {
  children: React.ReactNode
}) {
  return (
    <html lang="en">
      <head>
        {/* Bryntum Gantt 7.2.1 library loaded before page scripts */}
        {/* eslint-disable-next-line @next/next/no-sync-scripts */}
        <script src="/lib/gantt/gantt.umd.js" />
      </head>
      <body>
        <StoreProvider>{children}</StoreProvider>
      </body>
    </html>
  )
}

```

// app/login/page.tsx

```
import Link from 'next/link'
import LoginForm from '@/components/auth/LoginForm'

export default function LoginPage() {
  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-50">
      <div className="bg-white rounded-lg shadow p-8 w-full max-w-md">
        <h1 className="text-2xl font-bold text-gray-900 mb-6">登录</h1>
        <LoginForm />
        <p className="mt-4 text-center text-sm text-gray-600">
          没有账号？{' '}
          <Link href="/register" className="text-blue-600 hover:underline">
            立即注册
          </Link>
        </p>
      </div>
    </div>
  )
}

```

// app/page.tsx

```
import { redirect } from 'next/navigation'

```

```
export default function Home() {
  redirect('/login')
}

```

// app/projects/[id]/page.tsx

```
'use client'

import { useEffect, useRef, useState } from 'react'
import { useParams, useRouter } from 'next/navigation'
import dynamic from 'next/dynamic'
import { useAppDispatch, useAppSelector } from '@/store/hooks'
import { setTasks, clearTasks } from '@/store/slices/tasksSlice'
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

```
import { setCurrentProject } from '@/store/slices/projectSlice'
import GanttToolbar from '@/components/GanttChart/GanttToolbar'
import VersionPanel from '@/components/GanttChart/VersionPanel'
import Link from 'next/link'

const GanttChart = dynamic(() => import('@/components/GanttChart/GanttChart'), {
  ssr: false,
  loading: () => (
    <div className="flex items-center justify-center h-64 text-gray-500">
      Loading Gantt Chart...
    </div>
  ),
})

const COL_W_MIN  = 14
const COL_W_MAX  = 56
const COL_W_STEP = 7

export default function ProjectPage() {
  const params    = useParams()
  const projectId = params.id as string
  const dispatch  = useAppDispatch()
  const router    = useRouter()
  const { user }           = useAppSelector(s => s.auth)
  const { currentProject } = useAppSelector(s => s.project)
  const loadedRef = useRef(false)

  // ── Gantt UI state ──────────────────────────────────────────────
  const [colW,              setColW]              = useState(28)
  const [searchQuery,       setSearchQuery]       = useState('')
  const [expandAllSignal,   setExpandAllSignal]   = useState(0)
  const [collapseAllSignal, setCollapseAllSignal] = useState(0)
  const [focusSignal,       setFocusSignal]       = useState(0)
  const [showVersions,      setShowVersions]      = useState(false)

  useEffect(() => {
    if (!user) { router.push('/login'); return }
```

```
      if (loadedRef.current) return
      loadedRef.current = true

      fetch(`/api/projects/${projectId}`)
        .then(r => r.json())
        .then(data => { if (data.ok) dispatch(setCurrentProject(data.value)); else router.push('/dashboard') })

      fetch(`/api/tasks/${projectId}`)
        .then(r => r.json())
        .then(data => { if (data.ok) dispatch(setTasks(data.value)) })

      return () => {
        dispatch(clearTasks())
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

```
      dispatch(setCurrentProject(null))
    }
  }, [user, projectId, dispatch, router])

  if (!user) return null

  return (
    <div className="flex flex-col h-screen bg-white">
      <header className="bg-white border-b border-gray-200 px-6 py-3 flex items-center gap-4">
        <Link href="/dashboard" className="text-sm text-gray-500 hover:text-gray-700">
          ← 返回项目列表
        </Link>
        <h1 className="text-lg font-semibold text-gray-900">
          {currentProject?.name ?? '加载中...'}
        </h1>
      </header>

      <GanttToolbar
        projectId={projectId}
        colW={colW}
        onZoomIn={() => setColW(w => Math.min(COL_W_MAX, w + COL_W_STEP))}
        onZoomOut={() => setColW(w => Math.max(COL_W_MIN, w - COL_W_STEP))}
        onExpandAll={() => setExpandAllSignal(n => n + 1)}
        onCollapseAll={() => setCollapseAllSignal(n => n + 1)}
        onFocusTask={() => setFocusSignal(n => n + 1)}
        searchQuery={searchQuery}
        onSearchChange={setSearchQuery}
        onShowVersions={() => setShowVersions(true)}
      />

      {showVersions && (
        <VersionPanel projectId={projectId} onClose={() => setShowVersions(false)} />
      )}

      <div className="flex-1 overflow-hidden">
        <GanttChart
          projectId={projectId}
```

```
            statusDate={currentProject?.status_date}
            colW={colW}
            searchQuery={searchQuery}
            expandAllSignal={expandAllSignal}
            collapseAllSignal={collapseAllSignal}
            focusSignal={focusSignal}
          />
        </div>
      </div>
    )
  }

```

// app/register/page.tsx

```
import Link from 'next/link'
```

// app/api/auth/login/route.ts

// app/api/auth/logout/route.ts

// app/api/auth/register/route.ts

// app/api/dependencies/[projectId]/route.ts

// app/api/projects/[id]/route.ts

// app/api/projects/route.ts

// app/api/tasks/[projectId]/[taskId]/route.ts

// app/api/tasks/[projectId]/route.ts

// app/api/versions/[projectId]/[versionId]/route.ts

// app/api/versions/[projectId]/route.ts

// app/dashboard/page.tsx

// app/globals.css

// app/layout.tsx

// app/login/page.tsx

// app/page.tsx

// app/projects/[id]/page.tsx

// app/register/page.tsx

```
import RegisterForm from '@/components/auth/RegisterForm'

export default function RegisterPage() {
  return (
    <div className="min-h-screen flex items-center justify-center bg-gray-50">
      <div className="bg-white rounded-lg shadow p-8 w-full max-w-md">
        <h1 className="text-2xl font-bold text-gray-900 mb-6">创建账号</h1>
        <RegisterForm />
        <p className="mt-4 text-center text-sm text-gray-600">
          已有账号？{' '}
          <Link href="/login" className="text-blue-600 hover:underline">
            返回登录
          </Link>
        </p>
      </div>
    </div>
  )
}

```

// components/GanttChart/EditTaskModal.tsx

```
'use client'

import React, { useState, useEffect, useCallback } from 'react'
import { useAppDispatch, useAppSelector } from '@/store/hooks'
import { updateTasks, saveSnapshot } from '@/store/slices/tasksSlice'
import type { TaskLifecycleEvent } from '@/types'

interface Props {
  taskId: string
  projectId: string
  onClose: () => void
}
```

```
1482
1483  const EVENT_ICONS: Record<string, { icon: string; color: string }> = {
1484    created: { icon: '＋', color: 'text-green-600 bg-green-50' },
1485    updated: { icon: '✎',  color: 'text-blue-600 bg-blue-50'  },
1486    moved:   { icon: '⇅',  color: 'text-amber-600 bg-amber-50'},
1487    deleted: { icon: '－', color: 'text-red-500 bg-red-50'    },
1488  }
1489
1490  export default function EditTaskModal({ taskId, projectId, onClose }: Props) {
1491    const dispatch = useAppDispatch()
1492    const task = useAppSelector(s => s.tasks.tasks.find(t => t.id === taskId))
1493
1494    const [tab, setTab] = useState<'edit' | 'history'>('edit')
1495
1496    const [name,        setName]        = useState('')
1497    const [assignee,    setAssignee]    = useState('')
1498    const [startDate,   setStartDate]   = useState('')
1499    const [endDate,     setEndDate]     = useState('')
1500    const [percentDone, setPercentDone] = useState(0)
```